



Desenvolvimento Web

Objetos em JavaScript

Professor: Euclides Paim

euclidespaim@gmail.com



Desenvolvimento Web

Sumário

- **Objetos em JavaScript**
 - O que é um objeto — valores primitivos e valores compostos
 - Mutabilidade e manipulação **por referência**
 - Criação — literal, `new Object()`, função construtora e `Object.create()`
 - **Propriedades** — acesso, alteração, remoção e percurso
 - **Métodos** e a palavra-chave `this`
 - **Getters e setters** — encapsulamento e validação
 - **Construtores**, a palavra `new` e o `prototype`
 - **Classes** (ES6) e a comparação com a função construtora
- **Exercícios e Resumo**



Desenvolvimento Web

Introdução

- O que estudamos até agora?
 - **HTML** — elementos, atributos, classes, ids e formulários;
 - **CSS** — seletores, box model, posicionamento e responsividade;
 - **Fundamentos de JavaScript** — variáveis, operadores, tipos de dados, estruturas de controle e funções.
- Até aqui trabalhamos com dados **solto**s: cada informação em sua própria variável.
- A partir de agora vamos aprender a **agrupar dados e comportamentos** em uma única estrutura — é isso que chamamos de **objeto**.



Desenvolvimento Web

Por que precisamos de objetos?

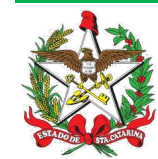
Imagine guardar os dados de um aluno usando apenas variáveis simples:

```
let alunoNome = "Ana Souza";  
let alunoIdade = 16;  
let alunoTurma = "202";
```

- Agora imagine **trinta alunos**. E que cada um também tenha e-mail, notas e frequência.
- O objeto resolve isso reunindo tudo o que descreve **uma mesma coisa**:

```
let aluno = { nome: "Ana Souza", idade: 16, turma: "202" };
```

- Uma variável, um conceito do mundo real, vários valores organizados **por nome**.



Objetos em JavaScript

O tipo de dado que organiza todos os outros



Desenvolvimento Web

O que é um objeto

O tipo de dado **fundamental** do JavaScript é o objeto.

- Um objeto é um **valor composto**: ele agrega vários valores — primitivos ou outros objetos — e permite **armazenar e recuperar** esses valores **por nome**.
- Cada par guardado no objeto tem duas partes:
 - a **chave** (ou nome da propriedade), sempre um texto ou um `Symbol`;
 - o **valor**, que pode ser de qualquer tipo — inclusive uma função.
- Por isso dizemos que um objeto JavaScript é uma **coleção de valores nomeados**, e que ela é **não ordenada**: não devemos depender da ordem em que as propriedades aparecem.



Desenvolvimento Web

Valores primitivos e valores compostos

O JavaScript tem **sete tipos primitivos**; todo o resto é objeto.

Característica	Primitivo	Objeto
Exemplos	string, number, boolean, null, undefined, symbol, bigint	objeto, array, função, data
Guarda	um único valor	vários valores nomeados
Pode ser alterado	não — é imutável	sim — é mutável
Atribuição copia	o valor	a referência
Comparação com ===	compara o valor	compara a referência

- Booleanos, números e textos também podem virar objetos quando criados com `new` — mas **isso não é recomendado**: deixa o código mais lento e gera comparações inesperadas.



Desenvolvimento Web

Objetos são mutáveis

Um valor primitivo **não muda**: quando alteramos a variável, ela passa a apontar para outro valor.

- Um objeto, ao contrário, pode ser alterado **depois de criado** — podemos acrescentar, mudar e remover propriedades a qualquer momento.

```
const pessoa = { nome: "Ana", idade: 16 };

pessoa.idade = 17;           // altera uma propriedade existente
pessoa.cidade = "Itajaí";    // acrescenta uma nova propriedade
delete pessoa.nome;          // remove uma propriedade
```

- Repare que o objeto foi declarado com `const` e ainda assim mudou. O `const` protege a **ligação da variável**, não o conteúdo do objeto: não podemos trocar o objeto inteiro, mas podemos mexer no que está dentro dele.



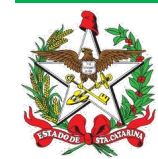
Desenvolvimento Web

Objetos são manipulados por referência

Ao atribuir um objeto a outra variável, o JavaScript **não faz uma cópia**: as duas variáveis passam a apontar para o **mesmo objeto** na memória.

```
const pessoa = { nome: "Ana", idade: 16 };  
const x = pessoa;      // NÃO cria uma cópia  
  
x.idade = 30;  
  
console.log(pessoa.idade); // 30 -> mudou também!
```

- `x` e `pessoa` são **o mesmo objeto**. Qualquer alteração feita por um aparece no outro.
- Para obter uma cópia de verdade usamos `Object.assign({}, pessoa)` ou `{ ...pessoa }` (operador de espalhamento).



Desenvolvimento Web

Comparando objetos

Como a comparação olha para a referência, dois objetos com o **mesmo conteúdo** não são considerados iguais:

```
const a = { nome: "Ana" };  
const b = { nome: "Ana" };  
const c = a;  
  
console.log(a === b);    // false -> objetos diferentes  
console.log(a === c);    // true  -> mesma referência
```

- É um dos erros mais comuns de quem está começando: === responde "**é o mesmo objeto?**", e não "**têm o mesmo conteúdo?**".



Criando objetos

Quatro caminhos para chegar ao mesmo lugar



Desenvolvimento Web

Formas de criar um objeto

Existem diferentes maneiras de criar novos objetos em JavaScript:

- definir e criar um único objeto usando um **literal de objeto**;
- definir e criar um único objeto com a palavra-chave `new`;
- definir um **construtor** e criar quantos objetos daquele tipo forem necessários;
- criar um objeto a partir de outro com `Object.create()`.

As duas primeiras produzem **um objeto de cada vez**. As duas últimas servem para produzir **muitos objetos com a mesma forma**.



Desenvolvimento Web

Forma 1 — literal de objeto

É a maneira mais simples e a mais usada. Definimos e criamos o objeto em **uma única instrução**.

- Um literal é uma lista de pares **nome: valor** separados por vírgula, entre chaves { }.

```
const pessoa = {  
  primeiroNome: "Ana",  
  ultimoNome: "Souza",  
  idade: 16,  
  corDosOlhos: "castanho"  
};
```

Os espaços e as quebras de linha são livres — o objeto acima poderia caber em uma linha só. Escrevemos em várias linhas apenas para **facilitar a leitura**.



Desenvolvimento Web

Forma 2 — a palavra-chave `new`

O exemplo a seguir cria um objeto com as mesmas quatro propriedades:

```
const pessoa = new Object();  
  
pessoa.primeiroNome = "Ana";  
pessoa.ultimoNome  = "Souza";  
pessoa.idade        = 16;  
pessoa.corDosOlhos  = "castanho";
```

- Os dois exemplos fazem exatamente a mesma coisa. **Não há necessidade** de usar `new Object()`.
- Por **simplicidade, legibilidade e velocidade de execução**, **prefira sempre o literal de objeto**.



Desenvolvimento Web

Forma 3 — função construtora

As duas formas anteriores criam **um objeto único**. Quando precisamos de muitos objetos do mesmo tipo, escrevemos um **molde**.

- Esse molde é uma **função construtora** — por convenção, com a inicial **maiúscula**:

```
function Pessoa(nome, idade, turma) {  
  this.nome = nome;  
  this.idade = idade;  
  this.turma = turma;  
}  
  
const aluna1 = new Pessoa("Ana", 16, "202");  
const aluna2 = new Pessoa("Bruna", 17, "301");
```

Obs.: Voltaremos a esse assunto com detalhe na seção de **construtores**.



Desenvolvimento Web

Forma 4 — *Object.create()*

Cria um objeto novo indicando explicitamente **qual objeto servirá de protótipo** para ele.

```
const base = {  
  apresentar() {  
    return "Olá, eu sou " + this.nome;  
  }  
};  
  
const aluna = Object.create(base);  
aluna.nome = "Ana";  
  
console.log(aluna.apresentar()); // Olá, eu sou Ana
```

- O objeto `aluna` **não tem** o método `apresentar`, mas consegue usá-lo: quando uma propriedade não é encontrada, o JavaScript a procura no **protótipo**. É a chamada **cadeia de protótipos**.



Propriedades

Os valores que o objeto guarda



Desenvolvimento Web

Propriedades do objeto

As propriedades são a parte mais **importante** de qualquer objeto JavaScript.

- Propriedades são os **valores associados a um objeto**. Um objeto é uma **coleção de propriedades não ordenadas**.
- Em geral elas podem ser **alteradas, adicionadas e excluídas**, mas algumas são somente leitura.
- Cada propriedade tem um nome e um valor:

Propriedade	Valor
nome	Ana Souza
idade	16
turma	202
ativa	true



Desenvolvimento Web

Acessando propriedades

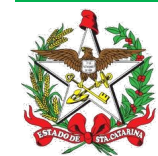
Há duas sintaxes para ler ou escrever uma propriedade:

```
objeto.propriedade      // notação de ponto
objeto["propriedade"]    // notação de colchetes
```

- O **ponto** é mais legível e deve ser a escolha padrão. Os **colchetes** são necessários quando:
 - o nome da propriedade está guardado em uma variável;
 - o nome tem espaço, hífen ou começa com número.

```
const aluna = { nome: "Ana", "nota final": 9.5 };

const campo = "nome";
console.log(aluna[campo]);           // Ana
console.log(aluna["nota final"]);    // 9.5
```



Desenvolvimento Web

Adicionando, alterando e removendo

Podemos **acrescentar** novas propriedades a um objeto já existente simplesmente atribuindo-lhes um valor:

```
const aluna = { nome: "Ana", idade: 16 };  
  
aluna.turma = "202";      // acrescenta  
aluna.idade = 17;         // altera
```

- A palavra-chave `delete` **exclui** uma propriedade do objeto:

```
delete aluna.idade;        // ou: delete aluna["idade"]  
  
console.log(aluna.idade);  // undefined
```

- O `delete` remove a propriedade inteira. **Não confunda** com atribuir `null`, que mantém a propriedade e apenas troca o valor.



Desenvolvimento Web

Objetos dentro de objetos

O valor de uma propriedade pode ser **outro objeto** ou um **array**. É assim que representamos informações mais complexas:

```
const aluna = {  
  nome: "Ana Souza",  
  endereco: {  
    cidade: "Itajaí",  
    bairro: "São Vicente"  
  },  
  notas: [8.5, 9.0, 7.5]  
};  
  
console.log(aluna.endereco.cidade); // Itajaí  
console.log(aluna.notas[0]);        // 8.5
```

- O acesso é encadeado, da esquerda para a direita. Se um nível intermediário não existir, ocorre erro — por isso existe o operador `?.` (`aluna.endereco?.cep`), que devolve `undefined` em vez de quebrar.



Desenvolvimento Web

Percorrendo as propriedades

Para visitar todas as propriedades de um objeto usamos `for...in` ou os métodos de `Object`:

```
const aluna = { nome: "Ana", idade: 16, turma: "202" };

for (const chave in aluna) {
  console.log(chave + ": " + aluna[chave]);
}

console.log(Object.keys(aluna)); // ["nome","idade","turma"]
console.log(Object.values(aluna)); // ["Ana", 16, "202"]
console.log(Object.entries(aluna)); // pares [chave, valor]
```

- **Atenção:** o `for...in` percorre também as propriedades **herdadas** do protótipo. Quando quisermos apenas as do próprio objeto, `Object.keys()` é a escolha mais segura.



Desenvolvimento Web

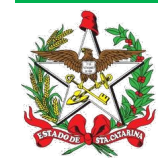
Verificando se uma propriedade existe

Três maneiras, com significados diferentes:

```
const aluna = { nome: "Ana", apelido: undefined };

"nome" in aluna           // true - existe (própria ou herdada)
Object.hasOwn(aluna, "nome") // true - existe no próprio objeto
aluna.apelido !== undefined // false - existe, mas vale undefined
```

- Comparar com `undefined` é o teste mais frágil: ele **não distingue** uma propriedade ausente de uma propriedade que existe e guarda `undefined`.
- `Object.hasOwn()` é a forma moderna; em código mais antigo aparece `aluna.hasOwnProperty("nome")`.



Métodos

O que o objeto sabe fazer



Desenvolvimento Web

Métodos do objeto

Métodos são **ações** que podem ser executadas sobre um objeto.

- As propriedades de um objeto podem guardar valores primitivos, outros objetos e **funções**.
- Um **método** nada mais é do que uma propriedade cujo valor é uma **definição de função**.

Propriedade	Valor
nome	Ana
sobrenome	Souza
idade	16
nomeCompleto	function() { return this.nome + " " + this.sobrenome; }

- Podemos dizer, então, que objetos JavaScript são **contêineres** para valores nomeados, chamados **propriedades** e **métodos**.



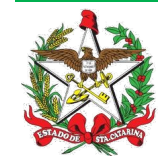
Desenvolvimento Web

Declarando e chamando métodos

Há duas sintaxes equivalentes. A **abreviada** é a preferida no código moderno:

```
const aluna = {  
  nome: "Ana",  
  sobrenome: "Souza",  
  
  // sintaxe longa  
  nomeCompleto: function () {  
    return this.nome + " " + this.sobrenome;  
  },  
  
  // sintaxe abreviada (ES6)  
  saudacao() {  
    return "Olá, " + this.nome + "!";  
  }  
};
```

- A chamada exige os **parênteses**: `aluna.nomeCompleto()` executa o método; `aluna.nomeCompleto` (sem parênteses) devolve a própria função.



Desenvolvimento Web

A palavra-chave `this`

Dentro de um método, `this` aponta para o **"dono" da chamada** — o objeto que está antes do ponto.

```
const aluna = {  
  nome: "Ana",  
  apresentar() {  
    return "Meu nome é " + this.nome;  
  }  
};  
  
aluna.apresentar();    // this é aluna -> "Meu nome é Ana"
```

- É o `this` que permite escrever **um único método** capaz de funcionar para qualquer objeto daquele tipo, sem repetir o nome da variável dentro da função.
- O valor de `this` é definido **no momento da chamada**, e não onde a função foi escrita.



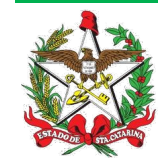
Desenvolvimento Web

this — a armadilha da arrow function

Funções de seta ($\Rightarrow$) **não possuem `this` próprio**: elas herdam o `this` do escopo onde foram escritas.

```
const aluna = {  
  nome: "Ana",  
  certo() {  
    return this.nome;           // "Ana"  
  },  
  errado: () => {  
    return this.nome;           // undefined  
  }  
};
```

- Regra prática: **use a sintaxe abreviada** para métodos de objeto e **evite arrow functions** quando o método precisar enxergar o próprio objeto.



Getters e Setters

Controlando o acesso às propriedades



Desenvolvimento Web

Encapsulamento — o problema

Os **getters** e **setters** são comuns em várias linguagens, e no JavaScript não é diferente: eles nos ajudam a **encapsular, proteger e isolar** propriedades.

- Normalmente os usamos quando precisamos fazer **validações** ou **tratamentos** antes de gravar um dado — ou ao recuperá-lo.
- Sem eles, o tratamento fica espalhado por todo o código que usa o objeto:

```
const produto = { valor: 123 };  
  
// quem usa o objeto precisa lembrar de formatar, toda vez  
console.log("R$ " + produto.valor.toFixed(2).replace(".", ","));
```

- E nada impede que alguém escreva `produto.valor = -50`.



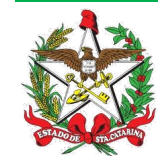
Desenvolvimento Web

get e set

Os manipuladores `get` e `set` foram incluídos no **ECMAScript 5**. Basta escrever o prefixo antes do nome da função:

```
const produto = {  
  valor: 0,  
  
  get moeda() { // leitura tratada  
    return "R$ " + this.valor.toFixed(2).replace(".", ",");  
  },  
  
  set preco(novo) { // escrita tratada  
    this.valor = novo;  
  }  
};  
  
produto.preco = 123; // sem parênteses!  
console.log(produto.moeda); // R$ 123,00
```

- Do lado de fora, `moeda` e `preco` **parecem propriedades comuns** — mas por trás existe uma função sendo executada.



Desenvolvimento Web

Validação dentro do setter

O maior ganho do setter é poder **recusar um valor inválido** antes que ele entre no objeto:

```
const aluna = {  
  _nota: 0,  
  
  get nota() {  
    return this._nota;  
  },  
  
  set nota(valor) {  
    if (valor < 0 || valor > 10) {  
      console.log("Nota inválida: " + valor);  
      return;  
    }  
    this._nota = valor;  
  }  
};  
  
aluna.nota = 9.5;    // aceito  
aluna.nota = 15;    // recusado
```



Desenvolvimento Web

Propriedade interna e campo privado

No exemplo anterior, `_nota` guarda o valor de verdade e `nota` é o acesso controlado.

- O sublinhado é apenas uma **convenção** entre programadores: ele **não impede** ninguém de escrever `aluna._nota = 99`.
- Para privacidade real, as **classes** oferecem campos privados com `#`:

```
class Aluna {  
    #nota = 0;                // realmente privado  
  
    get nota() { return this.#nota; }  
    set nota(v) { if (v >= 0 && v <= 10) this.#nota = v; }  
}  
  
const a = new Aluna();  
a.nota = 8;  
console.log(a.#nota);        // erro de sintaxe: fora da classe
```



Desenvolvimento Web

Object.defineProperty()

Permite criar uma propriedade descrevendo **como ela se comporta**:

```
const aluna = { nome: "Ana" };

Object.defineProperty(aluna, "matricula", {
  value: "2026001",
  writable: false,           // não pode ser alterada
  enumerable: false         // não aparece em for...in
});
```

Descritor	O que controla
writable	se o valor pode ser alterado
enumerable	se a propriedade aparece ao percorrer o objeto
configurable	se a propriedade pode ser removida ou reconfigurada

- Quando escrevemos uma propriedade da forma comum, os três descritores valem `true`.



Construtores

Um molde para produzir muitos objetos



Desenvolvimento Web

Função construtora

Os exemplos vistos até aqui são limitados: criam apenas **objetos únicos**. Às vezes precisamos de um **molde** para produzir muitos objetos do mesmo tipo.

- A maneira de criar um "tipo de objeto" é usar uma **função construtora**:

```
function Aluno(nome, idade, turma) {  
  this.nome = nome;  
  this.idade = idade;  
  this.turma = turma;  
  
  this.apresentar = function () {  
    return this.nome + ", turma " + this.turma;  
  };  
}  
  
const a1 = new Aluno("Ana", 16, "202");  
const a2 = new Aluno("Bruno", 17, "301");
```

- Por convenção o nome começa com **letra maiúscula** — é assim que avisamos que aquela função deve ser chamada com `new`.



Desenvolvimento Web

O que a palavra `new` faz

Quando escrevemos `new Aluno("Ana", 16, "202")`, o JavaScript executa quatro passos:

- 1. cria um objeto novo e vazio;
 - 2. liga esse objeto ao protótipo do construtor (`Aluno.prototype`);
 - 3. executa o corpo da função com `this` apontando para o objeto novo;
 - 4. devolve esse objeto automaticamente — sem precisar de `return`.
- Esquecer o `new` é um erro clássico: a função roda como função comum, `this` não aponta para um objeto novo e o resultado é `undefined`.
- As **classes** resolvem isso: chamar uma classe sem `new` gera erro imediatamente.



Desenvolvimento Web

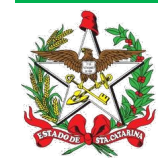
Métodos no construtor ou no prototype

No construtor da página anterior, **cada objeto criado ganha a sua própria cópia** do método `apresentar` — um desperdício quando há muitos objetos.

- Colocando o método no `prototype`, ele passa a existir **uma única vez**, compartilhado por todas as instâncias:

```
function Aluno(nome, turma) {  
  this.nome = nome;  
  this.turma = turma;  
}  
  
Aluno.prototype.apresentar = function () {  
  return this.nome + ", turma " + this.turma;  
};  
  
const a1 = new Aluno("Ana", "202");  
console.log(a1.apresentar()); // Ana, turma 202
```

- Regra prática: **dados** no construtor, **comportamento** no protótipo.



Classes

A sintaxe moderna sobre os protótipos



Desenvolvimento Web

Classes (ES6)

A partir do **ES6 (2015)** o JavaScript ganhou a palavra `class`, com uma escrita mais próxima da de outras linguagens:

```
class Aluno {  
  constructor(nome, idade, turma) {  
    this.nome = nome;  
    this.idade = idade;  
    this.turma = turma;  
  }  
  
  apresentar() {  
    // vai para o prototype  
    return this.nome + ", turma " + this.turma;  
  }  
}  
  
const a1 = new Aluno("Ana", 16, "202");  
console.log(a1.apresentar());
```

- Por baixo, continua sendo **protótipo**: a classe é **açúcar sintático** sobre o mesmo mecanismo.



Desenvolvimento Web

Classe com getters, setters e campo privado

```
class Produto {  
    #valor = 0;  
  
    constructor(nome, valor) {  
        this.nome = nome;  
        this.valor = valor;           // passa pelo setter  
    }  
  
    get valor() {  
        return this.#valor;  
    }  
  
    set valor(v) {  
        if (v < 0) throw new Error("Valor não pode ser negativo");  
        this.#valor = v;  
    }  
  
    get etiqueta() {  
        return this.nome + " - R$ " + this.#valor.toFixed(2);  
    }  
}
```



Desenvolvimento Web

Função construtora × classe

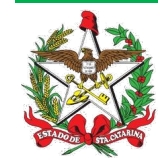
Aspecto	Função construtora	Classe (ES6)
Sintaxe	<code>function Aluno() { ... }</code>	<code>class Aluno { ... }</code>
Métodos compartilhados	<code>Aluno.prototype.metodo = ...</code>	escritos dentro do corpo
Chamada sem <code>new</code>	executa e devolve <code>undefined</code>	gera erro imediatamente
Uso antes de declarar	permitido (içamento)	não permitido
Campos privados	apenas por convenção (<code>_nome</code>)	reais, com <code>#nome</code>
Getters e setters	com <code>Object.defineProperty</code>	<code>get</code> e <code>set</code> no corpo

- As duas formas produzem objetos equivalentes. A **classe** é hoje a escrita recomendada; a **função construtora** ainda aparece em muito código existente, e por isso precisamos saber lê-la.



Exercícios

Hora de escrever código



Desenvolvimento Web

Exercício 1 — O objeto *livro*

Crie um objeto `livro` usando um literal, com as propriedades **titulo**, **autor**, **ano** e **paginas**.

- Requisitos:
 - exiba o título e o autor no console usando a notação de ponto;
 - acrescente a propriedade **editora** depois de criar o objeto;
 - remova a propriedade **paginas** com `delete`;
 - percorra o objeto com `for...in` mostrando cada par chave/valor.

Desafio: acrescente um método `ficha()` que devolva uma frase com todos os dados.



Desenvolvimento Web

Exercício 2 — Conta bancária com validação

Crie um objeto `conta` com uma propriedade interna `_saldo` iniciada em zero.

- Requisitos:
 - um **getter** `saldo` que devolva o valor formatado como R\$ 0,00;
 - um **setter** `deposito` que só aceite valores **maiores que zero**;
 - um método `sacar(valor)` que recuse o saque se não houver saldo suficiente;
 - mensagens no console para cada operação recusada.

Desafio: guarde todas as operações em um array `extrato` e crie um método para exibi-lo.



Desenvolvimento Web

Exercício 3 — Construtor e classe

Escreva uma **função construtora** `Aluno(nome, turma)` com um método `apresentar()` no prototype.

- Em seguida, **reescreva o mesmo código usando `class`**.
- Requisitos:
 - criar pelo menos três alunos e exibir a apresentação de cada um;
 - guardar os três objetos em um array e percorrê-lo com `for...of`;
 - acrescentar à classe um campo privado `#notas` e um método `media()`.

Desafio: escreva um comentário explicando o que muda entre as duas versões.



Desenvolvimento Web

Exercício 4 — Desafio: catálogo de produtos

Monte um pequeno catálogo em memória, sem usar banco de dados.

- Requisitos:
 - uma classe `Produto` com `nome`, `preco` (privado, com validação) e `estoque`;
 - um array `catalogo` com pelo menos cinco produtos;
 - um getter `etiqueta` que devolva nome e preço formatados;
 - funções para **buscar por nome**, **listar os esgotados** e **calcular o valor total** do estoque.

Desafio extra: exiba o catálogo em uma tabela HTML, usando o que já sabemos de manipulação do DOM.



Desenvolvimento Web

Resumo

Objetos em JavaScript

- O objeto é um **valor composto**: uma coleção não ordenada de valores nomeados
- É **mutável** e manipulado **por referência** — `===` compara a referência, não o conteúdo
- Criação: **literal** (preferido), `new Object()`, **função construtora** e `Object.create()`
- **Propriedades**: acesso por ponto ou colchetes; `delete` remove; `Object.keys()` percorre com segurança
- **Métodos** são propriedades que guardam funções; `this` aponta para o dono da chamada
- Arrow functions **não têm `this` próprio** — evite-as como métodos de objeto
- **Getters e setters** permitem validar e formatar sem mudar quem usa o objeto
- A palavra `new` cria o objeto, liga o protótipo, executa o construtor e o devolve
- **Dados** no construtor, **comportamento** no `prototype`
- **Classes** são a escrita moderna sobre o mesmo mecanismo de protótipos, com campos privados `#`

Exercícios — livro, conta bancária, construtor × classe e catálogo de produtos



Referências

Referências Básicas

FLANAGAN, David. JavaScript: o guia definitivo. 6. ed. Porto Alegre: Bookman, 2012.

RESIG, John; BIBEAULT, Bear; MARZ, Josip. Secrets of the JavaScript Ninja. 2. ed. Shelter Island: Manning, 2016.

GRONER, Loiane. Estrutura de dados e algoritmos com JavaScript. São Paulo: Novatec, 2019.

Referências Complementares

SILVA, Maurício Samy. HTML 5: a linguagem de marcação que revolucionou a web. São Paulo: Novatec, 2011.

IEPSEN, Edécio Fernandes. Lógica de Programação e Algoritmos com JavaScript. São Paulo: Novatec, 2018.

SIMPSON, Kyle. You Don't Know JS: this & Object Prototypes. Sebastopol: O'Reilly, 2014.

Referências na Internet

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Working_with_objects

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Classes>

https://www.w3schools.com/js/js_objects.asp

<https://javascript.info/object>